

## **Seminar 2**

# **Grafuri Euleriene**

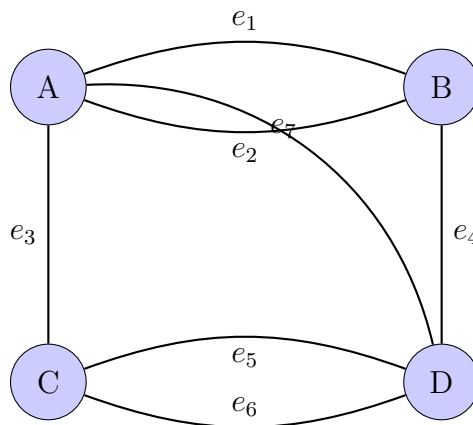
Asist. univ. dr. Mihai Tudor

21 februarie 2026

# 1 Motivație istorică

Problema grafurilor euleriene a apărut din celebra **Problema podurilor din Königsberg** (1736). Orașul Königsberg (astăzi Kaliningrad) avea 7 poduri care conectau 4 zone de uscat. S-a pus întrebarea : este posibil să parcurgem toate cele 7 poduri exact o dată și să revenim în punctul de plecare?

Leonhard Euler a demonstrat că acest lucru este imposibil și a formulat condiții generale pentru existența unor astfel de trasee.



În graful de mai sus:

- $\deg(A) = 5$  (impar)
- $\deg(B) = 3$  (impar)
- $\deg(C) = 3$  (impar)
- $\deg(D) = 3$  (impar)

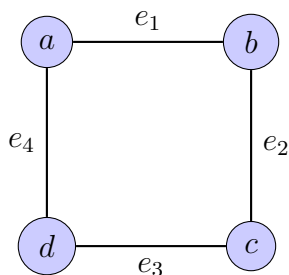
Toate vârfurile au grad impar! Euler a descoperit că aceasta este problema fundamentală.

## 2 Definiții fundamentale

**Definiție 2.1.** Fie  $G = (V, E)$  un graf neorientat.

- Un **drum eulerian** (sau **lanț eulerian**) este un drum care trece prin fiecare muchie exact o dată.
- Un **ciclu eulerian** (sau **circuit eulerian**) este un ciclu care trece prin fiecare muchie exact o dată și se întoarce la vârful de start.
- Un graf care admite un ciclu eulerian se numește **graf eulerian**.
- Un graf care admite un drum eulerian (dar nu ciclu) se numește **semi-eulerian**.

**Exemplu 2.1.** Graf eulerian simplu (pătrat):



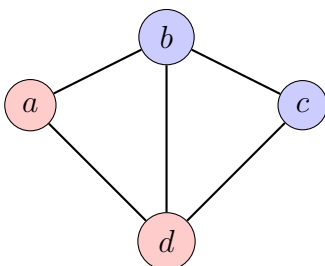
Grade:  $\deg(a) = 2$ ,  $\deg(b) = 2$ ,  $\deg(c) = 2$ ,  $\deg(d) = 2$

Toate vârfurile au grad par!

Un ciclu eulerian posibil:  $a \rightarrow b \rightarrow c \rightarrow d \rightarrow a$

Acest drum parcurge muchiile  $e_1, e_2, e_3, e_4$  exact o dată fiecare.

**Exemplu 2.2.** Graf semi-eulerian (are drum eulerian, dar nu ciclu):



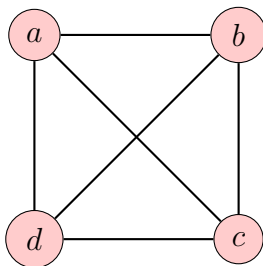
Grade:  $\deg(a) = 2$  (par),  $\deg(b) = 3$  (impar, roșu),  $\deg(c) = 2$  (par),  $\deg(d) = 3$  (impar, roșu)

Exact 2 vârfuri cu grad impar!

Un drum eulerian:  $b \rightarrow a \rightarrow d \rightarrow c \rightarrow b \rightarrow d$  (start în  $b$ , final în  $d$ )

Nu există ciclu eulerian (nu putem reveni la punctul de plecare după ce parcurgem toate muchiile).

**Exemplu 2.3.** Graf care NU este nici eulerian, nici semi-eulerian:



Grade:  $\deg(a) = 3$ ,  $\deg(b) = 3$ ,  $\deg(c) = 3$ ,  $\deg(d) = 3$

Are 4 vârfuri cu grad impar! NU este nici eulerian, nici semi-eulerian.

Nu putem parcurge toate muchiile exact o dată.

### 3 Teorema lui Euler

**Teoremă 3.1** (Teorema lui Euler pentru cicluri euleriene). *Un graf conex  $G = (V, E)$  admite un ciclu eulerian dacă și numai dacă toate vârfurile au grad par.*

**Teoremă 3.2** (Teorema lui Euler pentru drumuri euleriene). *Un graf conex  $G = (V, E)$  admite un drum eulerian (dar nu ciclu) dacă și numai dacă are exact două vârfuri cu grad impar.*

**Observație 3.1.** *Putem sintetiza cele două rezultate:*

- **0 vârfuri cu grad impar**  $\Rightarrow$  graf eulerian (există ciclu eulerian)
- **Exact 2 vârfuri cu grad impar**  $\Rightarrow$  graf semi-eulerian (există drum eulerian, nu ciclu)
- **Mai mult de 2 vârfuri cu grad impar**  $\Rightarrow$  NU există drum eulerian

**Exemplu 3.1.** Verificare în Python - determinare tip graf:

```

1 def calculeaza_grade(graf):
2     """Calculeaza gradul fiecarui varf"""
3     grade = {}
4     for varf in graf:
5         grade[varf] = len(graf[varf])
6     return grade
7
8 def tip_graf_eulerian(graf):
9     """
10    Determina daca un graf este eulerian, semi-eulerian sau
11    niciunul.
12    Returneaza: 'eulerian', 'semi-eulerian' sau 'nu este eulerian'
13    """
14    grade = calculeaza_grade(graf)
15
16    # Numaram varfurile cu grad impar
17    varfuri_impere = [v for v in grade if grade[v] % 2 == 1]
18    nr_impere = len(varfuri_impere)
19
20    if nr_impere == 0:
21        return 'eulerian', varfuri_impere
22    elif nr_impere == 2:
23        return 'semi-eulerian', varfuri_impere
24    else:
25        return 'nu este eulerian', varfuri_impere
26
27 # Test 1: Patrat (eulerian)
28 patrat = {
29     'a': ['b', 'd'],
30     'b': ['a', 'c'],
31     'c': ['b', 'd'],
32     'd': ['c', 'a']
33 }
34
35 tip, varfuri = tip_graf_eulerian(patrat)
36 print(f"Patrat: {tip}")
37 print(f"Varfuri cu grad impar: {varfuri}\n")

```

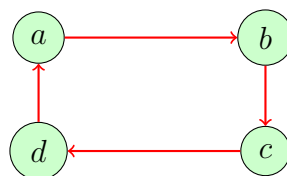
```

37
38 # Test 2: Graf semi-eulerian
39 semi = {
40     'a': ['b', 'd'],
41     'b': ['a', 'c', 'd'],
42     'c': ['b', 'd'],
43     'd': ['a', 'b', 'c']
44 }
45
46 tip, varfuri = tip_graf_eulerian(semi)
47 print(f"Graf semi: {tip}")
48 print(f"Varfuri cu grad impar: {varfuri}\n")
49
50 # Test 3: Podurile din Konigsberg
51 konigsberg = {
52     'A': ['B', 'B', 'C', 'D'],
53     'B': ['A', 'A', 'D'],
54     'C': ['A', 'D', 'D'],
55     'D': ['A', 'B', 'C', 'C']
56 }
57
58 tip, varfuri = tip_graf_eulerian(konigsberg)
59 print(f"Konigsberg: {tip}")
60 print(f"Varfuri cu grad impar: {varfuri}")
61
62 # Output:
63 # Patrat: eulerian
64 # Varfuri cu grad impar: []
65 #
66 # Graf semi: semi-eulerian
67 # Varfuri cu grad impar: ['b', 'd']
68 #
69 # Konigsberg: nu este eulerian
70 # Varfuri cu grad impar: ['A', 'B', 'C', 'D']

```

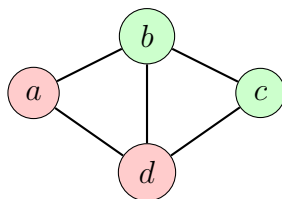
**Exemplu 3.2.** Exemple vizuale pentru fiecare caz:

**Caz 1: Graf eulerian (toate vârfurile au grad par)**



Ciclu:  $a \rightarrow b \rightarrow c \rightarrow d \rightarrow a$   
 Grade: toate par (2, 2, 2, 2)

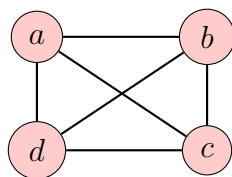
**Caz 2: Graf semi-eulerian (exact 2 vârfuri cu grad impar)**



*Drum: start în a sau c (grad impar)*

*Grade:  $a=3, b=3, c=2, d=2$*

**Caz 3: Graf ne-eulerian (mai mult de 2 vârfuri cu grad impar)**



*NU există drum eulerian*

*Grade: toate impare (3, 3, 3, 3)*

## 4 Găsirea unui ciclu/drum eulerian

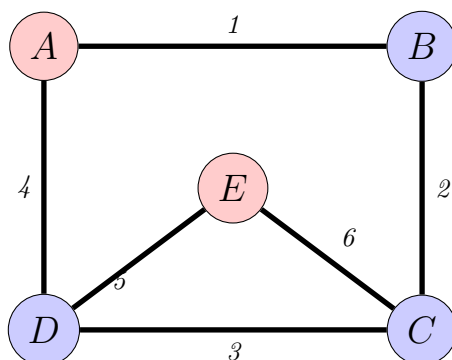
### 4.1 Algoritmul lui Hierholzer (simplificat)

Pentru a găsi efectiv un ciclu eulerian, folosim **Algoritmul lui Hierholzer**:

1. Pornim dintr-un vârf arbitrar (pentru ciclu) sau dintr-un vârf cu grad impar (pentru drum)
2. Construim un ciclu parțial urmând muchiile nevizitate
3. Când ne întoarcem la start și mai avem muchii nevizitate, pornim un nou ciclu din orice vârf care mai are muchii
4. Combinăm ciclurile într-un singur ciclu eulerian

**Exemplu 4.1.** Considerăm următorul graf semi-eulerian (are exact 2 vârfuri cu grad impar):

**GRAF ÎNIȚIAL:**

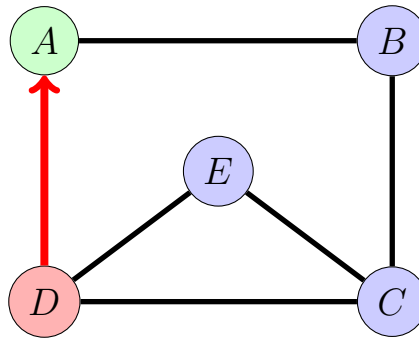


| Vârf | Grad                    |
|------|-------------------------|
| A    | 2 (par)                 |
| B    | 2 (par)                 |
| C    | 3 (impar) - <i>roșu</i> |
| D    | 3 (impar) - <i>roșu</i> |
| E    | 2 (par)                 |

**Concluzie:** 2 vârfuri cu grad impar  $\Rightarrow$  graf semi-eulerian

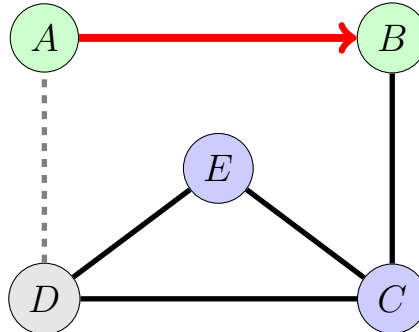
Drumul eulerian trebuie să înceapă din D și să termine în C (sau invers).

**PAS 1:** Start din D (grad impar). Alegem muchia 4:  $D \rightarrow A$



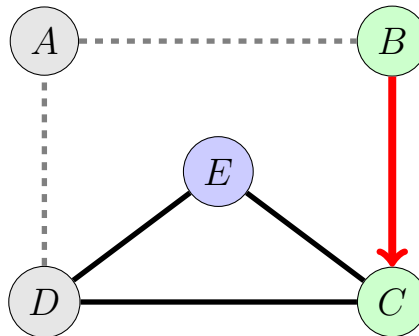
Drum:  $D \rightarrow A$

**PAS 2:** Din A, alegem muchia 1:  $A \rightarrow B$



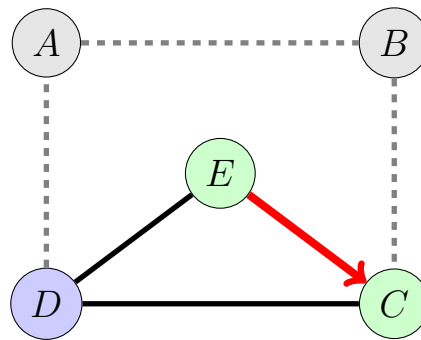
Drum:  $D \rightarrow A \rightarrow B$

**PAS 3:** Din B, alegem muchia 2:  $B \rightarrow C$



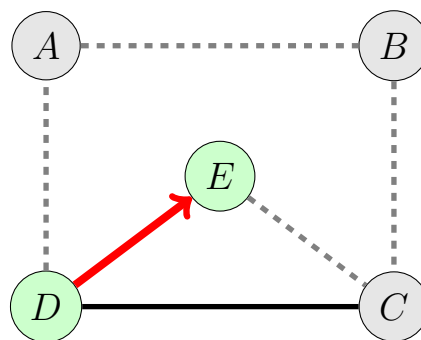
Drum:  $D \rightarrow A \rightarrow B \rightarrow C$

**PAS 4:** Din  $C$ , alegem muchia 6:  $C \rightarrow E$



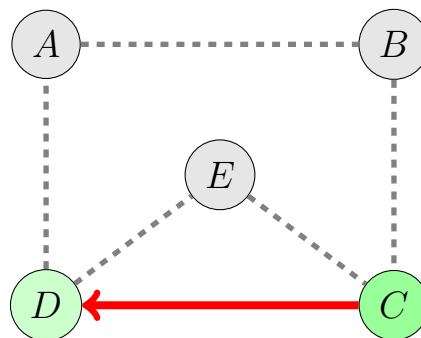
Drum:  $D \rightarrow A \rightarrow B \rightarrow C \rightarrow E$

**PAS 5:** Din  $E$ , alegem muchia 5:  $E \rightarrow D$



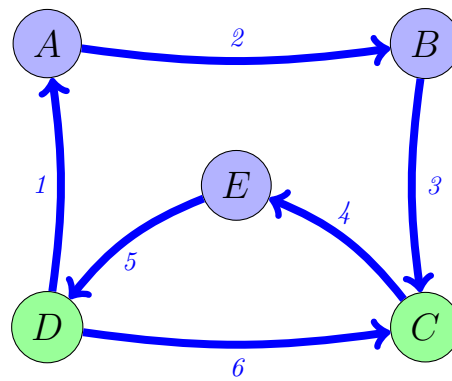
Drum:  $D \rightarrow A \rightarrow B \rightarrow C \rightarrow E \rightarrow D$

**PAS 6:** Din  $D$ , alegem ultima muchie 3:  $D \rightarrow C$  (FINAL!)



Drum:  $D \rightarrow A \rightarrow B \rightarrow C \rightarrow E \rightarrow D \rightarrow C$

**REZULTAT FINAL:**



**DRUM EULERIAN:**  $D \rightarrow A \rightarrow B \rightarrow C \rightarrow E \rightarrow D \rightarrow C$

**Verificare:**

- Start: D (grad impar)
- Final: C (grad impar)
- Muchii parcurse: 4, 1, 2, 6, 5, 3 - toate cele 6 muchii exact o dată!

**Exemplu 4.2.** Implementare simplificată în Python:

```

1 def gaseste_ciclu_eulerian_simplu(graf):
2     """
3     Gaseste un ciclu eulerian (versiune simplificata).
4     Presupunem ca graful ESTE eulerian (nu verificam).
5     """
6     # Copiem graful (il vom modifica)
7     from copy import deepcopy
8     g = deepcopy(graf)
9
10    # Pornim dintr-un varf oarecare
11    start = list(g.keys())[0]
12    ciclu = []
13    stiva = [start]
14
15    while stiva:
16        varf = stiva[-1]
17
18        if g[varf]: # Mai are vecini nevizitati
19            # Alegem un vecin
20            vecin = g[varf][0]
21
22            # Stergem muchia in ambele directii (graf neorientat)
23            g[varf].remove(vecin)
24            g[vecin].remove(varf)
25
26            # Continuam din vecin
27            stiva.append(vecin)
28        else:
29            # Nu mai are vecini, adaugam in ciclu

```

```

30         ciclu.append(stiva.pop())
31
32     return ciclu[::-1] # Inversam pentru ordinea corecta
33
34 # Test pe patrat
35 patrat = {
36     'a': ['b', 'd'],
37     'b': ['a', 'c'],
38     'c': ['b', 'd'],
39     'd': ['a', 'c']
40 }
41
42 ciclu = gaseste_ciclu_eulerian_simplu(patrat)
43 print("Ciclu eulerian pentru patrat:")
44 print(" -> ".join(ciclu))
45 # Output posibil: a -> b -> c -> d -> a
46
47 # Test pe graf mai complex
48 graf_complex = {
49     1: [2, 3],
50     2: [1, 3, 4],
51     3: [1, 2, 4],
52     4: [2, 3]
53 }
54
55 ciclu = gaseste_ciclu_eulerian_simplu(graf_complex)
56 print("\nCiclu eulerian pentru graf complex:")
57 print(" -> ".join(map(str, ciclu)))

```

**Exemplu 4.3.** Găsire drum eulerian (pentru graf semi-eulerian):

```

1 def gaseste_drum_eulerian(graf):
2     """
3     Gaseste drum eulerian pentru graf semi-eulerian.
4     """
5     from copy import deepcopy
6
7     # Gasim varfurile cu grad impar (start si final)
8     grade = {v: len(vecini) for v, vecini in graf.items()}
9     varfuri_impere = [v for v in grade if grade[v] % 2 == 1]
10
11     if len(varfuri_impere) != 2:
12         return None # Nu e semi-eulerian
13
14     # Copiem graful
15     g = deepcopy(graf)
16
17     # Start din primul varf cu grad impar
18     start = varfuri_impere[0]
19     drum = []
20     stiva = [start]
21

```

```

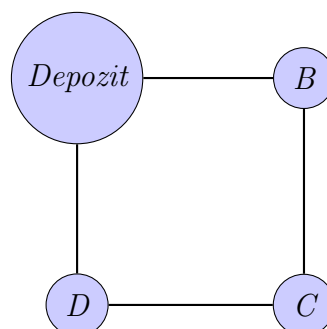
22     while stiva:
23         varf = stiva[-1]
24
25         if g[varf]:
26             vecin = g[varf][0]
27             g[varf].remove(vecin)
28             g[vecin].remove(varf)
29             stiva.append(vecin)
30         else:
31             drum.append(stiva.pop())
32
33     return drum[::-1]
34
35 # Test
36 graf_semi = {
37     'a': ['b'],
38     'b': ['a', 'c', 'd'],
39     'c': ['b', 'd'],
40     'd': ['b', 'c']
41 }
42
43 drum = gaseste_drum_eulerian(graf_semi)
44 if drum:
45     print("Drum eulerian:")
46     print(" -> ".join(drum))
47     print(f"Start: {drum[0]}, Final: {drum[-1]}")
48 else:
49     print("Nu exista drum eulerian")
50
51 # Output:
52 # Drum eulerian:
53 # a -> b -> c -> d -> b
54 # Start: a, Final: b

```

## 5 Aplicații practice

### Exemplu 5.1. Aplicație 1: Optimizarea rutelor de livrare

Un curier trebuie să parcurgă toate străzile dintr-un cartier (muchiiile) exact o dată. Vrem să știm dacă poate face asta și să revină la depozit.



```

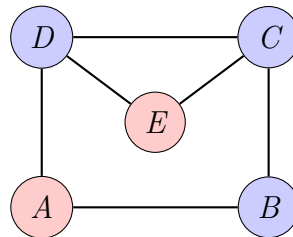
1 # Modelam cartierul ca graf
2 cartier = {
3     'Depozit': ['B', 'D'],
4     'B': ['Depozit', 'C'],
5     'C': ['B', 'D'],
6     'D': ['C', 'Depozit']
7 }
8
9 rezultat = verifica_eulerian(cartier)
10 print(f"Tip: {rezultat['tip']}")
11 print(f"Mesaj: {rezultat['mesaj']}")
12
13 if rezultat['tip'] == 'eulerian':
14     ruta = gaseste_ciclu_eulerian_simplu(cartier)
15     print(f"\nRuta optima: {' -> '.join(ruta)}")
16     print("Curierul poate reveni la depozit!")

```

### Exemplu 5.2. Aplicație 2: Desenare fără ridicare creionului

Putem desena o figură fără să ridicăm creionul de pe hârtie?

Figura în formă de plic:



```

1 plic = {
2     'A': ['B', 'D'],
3     'B': ['A', 'C'],
4     'C': ['B', 'D', 'E'],
5     'D': ['A', 'C', 'E'],
6     'E': ['C', 'D']
7 }
8
9 rezultat = verifica_eulerian(plic)
10 print(f"Tip: {rezultat['tip']}")
11 print(f"Grade: {rezultat['grade']}")
12
13 if rezultat['tip'] == 'semi-eulerian':
14     print(f"\nPutem desena fara sa ridicam creionul!")
15     print(f"Start din {rezultat['start']} si terminam in {rezultat['final']}")
16     drum = gaseste_drum_eulerian(plic)
17     print(f"Ordinea: {' -> '.join(drum)}")
18 elif rezultat['tip'] == 'eulerian':
19     print("\nPutem desena si reveni la punctul de start!")
20 else:
21     print("\nNU putem desena fara sa ridicam creionul")

```

**Exemplu 5.3. Aplicație 3: Problema podurilor**

Revenind la Königsberg - putem verifica de ce problema nu are soluție:

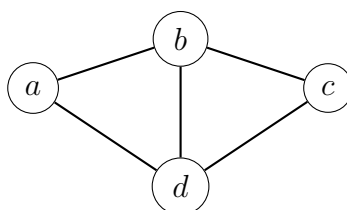
```

1 # Modelarea originala a Königsberg
2 # (simplificat fara muchii multiple)
3 konigsberg_simplificat = {
4     'Nord': ['Est', 'Vest', 'Insula1', 'Insula2'],
5     'Sud': ['Est', 'Vest', 'Insula1', 'Insula2'],
6     'Est': ['Nord', 'Sud'],
7     'Vest': ['Nord', 'Sud'],
8     'Insula1': ['Nord', 'Sud'],
9     'Insula2': ['Nord', 'Sud']
10 }
11
12 rezultat = verifica_eulerian(konigsberg_simplificat)
13 print("Problema podurilor din Königsberg:")
14 print(f"Tip: {rezultat['tip']}")
15 print(f"Mesaj: {rezultat['mesaj']}")
16
17 if 'varfuri_impere' in rezultat:
18     print(f"Varfuri cu grad impar: {rezultat['varfuri_impere']}")
19     print(f"Numar varfuri impere: {len(rezultat['varfuri_impere'])}")
20
21 print("\nConcluziei lui Euler: ")
22 print("Este IMPOSIBIL sa parcurgem toate podurile exact o data!")

```

## 6 Exerciții propuse

1. Verificați dacă următorul graf este eulerian, semi-eulerian sau niciunul:



2. Desenați un graf cu 6 vârfuri care să fie eulerian.
3. Desenați un graf cu 5 vârfuri care să fie semi-eulerian (dar nu eulerian).
4. Demonstrați că dacă un graf are mai mult de două vârfuri cu grad impar, atunci nu poate fi semi-eulerian.
5. Scrieți un program Python care citește un graf și determină:
  - Dacă este eulerian/semi-eulerian
  - Găsește și afișează un ciclu/drum eulerian dacă există

6. Considerați un oraș cu 8 intersecții și 12 străzi. Cum ar trebui să fie conectate străzile pentru ca un serviciu de salubritate să poată parcurge toate străzile exact o dată și să revină la depozit?
7. Găsiți un exemplu de graf care este atât eulerian, cât și hamiltonian.
8. Este posibil ca un graf regulat (toate vârfurile au același grad) cu grad impar să fie eulerian? Justificați.
9. Modificați problema podurilor din Königsberg: dacă am putea adăuga un singur pod, unde ar trebui plasat pentru ca problema să aibă soluție?
10. Implementați algoritmul lui Hierholzer complet în Python și testați-l pe diferite grafuri.